

Syntax and semantics: **Syntax** is the structure (rules for writing code), **Semantics** is the meaning (what the code does. Ex- does 'x + y' add numbers or concatenate strings?).

x = y + 1 → syntax rules
= means assignment → semantics

Language paradigms: Ways of organizing code, like imperative (step-by-step) or functional (math-like functions).

Static vs. dynamic typing: Static checks variable types at compile time (e.g., Java); dynamic at runtime (e.g., Python).

Compilation vs. interpretation: Compilation translates code to machine code upfront, interpretation executes line-by-line.

Programming Languages: formal, structured languages used to give instructions to a computer. They allow humans to tell a computer what to do and how to do it. A set of **Rules, Symbols, Syntax, Semantics** Used to write instructions that a computer can execute.

Why Do We Need Programming Languages? Computers understand only binary (0s and 1s). Programming languages act as a bridge between Human thinking and Machine instructions (Translate high level ideas into low level operations).

Types of Programming Languages: **Low-Level Languages:** Closest to hardware. Ex- **Machine language** (Binary 1/0), **Assembly language** (Uses mnemonics like MOV AL, 61h (human-readable version of binary). **Needs an assembler** to convert to machine code.). **Advantages of LL:** Very fast, Direct hardware control. **Disadvantages of LL** -Difficult to write, Error-prone, Not portable.

High-Level Languages: Looks close to English. Focuses on logic, not hardware details. **Advantages of HL:** Easy to read & write, Portable (runs on any machine with proper compiler/interpreter/VM). Developer-friendly **Disadvantages of HL:** Slightly slower than low-level languages, Overhead due to translation, Modern optimizations reduce this overhead.

Programming Paradigms

Imperative Programming Paradigm	Declarative Programming Paradigm
Procedural programming Paradigm	Logic Programming Paradigm
Object Oriented Programming	Functional Programming
Parallel Processing Approach	Database Processing Approach

Paradigm-Based Types: Programming Paradigm-Defines the style of programming, Describes how problems are solved.

Imperative Languages (Paradigm): Focus on how to do things. Use variables, assignments, loops. Change program state. Examples: C, Python, Java. **Advantages:** Very simple to implement..It contains loops, variables etc. **Disadvantages:** Complex problem cannot be solved (due to frequent state change), Less efficient and less productive, Parallel programming is not possible (due to shared mutable state). **Procedural Programming paradigm:** Programs organized into **procedures/functions**. Reusable blocks of code. Example: C, Pascal. **Advantages:** reusability. **Object-Oriented Languages:** Combine data + behavior. Use classes, objects, methods. Support encapsulation, inheritance. Examples: Java, C++, Python. **Advantages:** Data security, Inheritance, Code reusability. **Declarative Programming Paradigm:** What result is required, not how to compute it. **No explicit control flow: No loops, step-by-step instructions.** Easier parallelism: Fewer dependencies on execution order. **Advantages:** Simplifies program understanding. Easier to write parallel programs Because there is little or no shared mutable state. Easier reasoning and debugging. **Functional Languages:** Focus on mathematical functions. No side effects. Immutability (avoid changing state). Use recursion, higher-order functions. **Loose coupling:** data is passed to functions, not tightly bound. Examples: Haskell, Lisp, Scala. **Logic Programming:** Based on rules and inference. Program states what is true, not how to compute. System finds the solution. Based on a knowledge base (facts + rules). Widely used in AI and reasoning systems. Example: Prolog.

Scripting Languages: Used for automation and rapid development. Often interpreted. Examples: Python, JavaScript. **Parallel Processing:** Multiple computations run **simultaneously**, Improves performance (Executing a program by **dividing instructions across multiple processors**).

Concurrent not true parallelism (interlaved). **Uses multi-core CPUs.** **Concurrency models.** **Threads & Locks.** **Message Passing.** **Actor Model.** **Challenges**-Shared state, **Race conditions**, **Synchronization issues**

Database Programming: Focus on **data storage & retrieval**. Queries describe *what* data is needed. focus on Data and its movement, not step-by-step procedures. Program behavior is **defined by data**, not hard-coded logic. Uses **queries** to specify *what data is needed* Example: SQL.

Code Translation to machine code: Compiler → translates the whole program before execution. Produces and executable file (C, C++). **Interpreter** → translates & executes line by line (no separate executable file (Python, Ruby). **Hybrid** → compiled to bytecode, run on VM (Java, C#).

Why Study Programming Languages? Pick the right language/tool for a task. Learn trade-offs in language design. Write better, safer, efficient programs. Language-agnostic engineers.

Language Design Goals: **Readability**-Easy to read and understand code. **Writability**-Easy to write programs quickly. **Reliability**-Reduces errors, behaves predictably. **Efficiency**-Uses time and memory effectively. **Portability**-Runs on multiple platforms.

Language Evaluation Criteria: **Simplicity**-Few basic constructs. Easy to learn and use. **Orthogonality**-Features work consistently together. Fewer special cases. **Abstraction** **Support**-Support for functions, modules, classes, Helps manage complexity. **Security**-Prevents unsafe operations. Protects data and execution.

Can language be both compiler and interpreter?: Yes. A programming language is only a **specification** (rules, syntax, semantics). Whether it is **compiled or interpreted** depends on the **implementation**, not the language itself.

Constructs: valid piece of language uses to write programs. Ex- IF else, for, while

Function names, try-Catch.

Lexical Elements: *Smallest meaningful units* of a programming language.

Tokens: Basic lexical units recognized by the compiler. **Common token types:** **Identifiers** → names of variables, functions, classes. **Keywords** → reserved words with predefined meaning (if, while). **Operators** → symbols for operations (+, -, *, **Punctuators** → structure symbols (;, {}, (), **Phases of a compiler/interpreter:** **Lexical Analysis** (first phase) and **Syntax Analysis** (second phase). there are many more, input/output of each phase is marked in diagram.

Flowchart of Compiler Phases:

```
graph TD
    A[CHARACTER STREAM] --> B[LEXICAL ANALYSER]
    B -- "TOKEN STREAM" --> C[SYNTAX ANALYSER]
    C -- "SYNTAX TREE" --> D[SEMANTIC ANALYSER]
    D -- "SYNTAX TREE" --> E[INTERMEDIATE CODE GENERATOR]
    E -- "INTERMEDIATE CODE" --> F[CODE OPTIMIZER]
    F -- "OPTIMIZED INTERMEDIATE CODE" --> G[CODE GENERATOR]
    G --> H[TARGET MACHINE CODE]
```

Front End: Checks correctness of the program (Lexical, Syntax, Semantic).

Back End: Focus on efficiency (Intermediate Code Generator, Code Optimizer, Code Generator).

Intermediate Code Generator: (machine -independent intermediate code. Converts syntax tree into intermediate representation (IR), Makes compiler portable.

Diagram of Intermediate Code Generator:

```
graph TD
    A["<id,1> = <id,2> * <id,3> <br> 60"] --> B[INTERMEDIATE CODE GENERATOR]
    B --> C["t1 = inttofloat(60)  
t2 = id3 * t1  
t3 = id2 * t2  
id1 = t3"]
    C --> D[CODE OPTIMIZER]
    D --> E["t1 = id3 * 60.0  
id1 = id2 * t1"]
    E --> F[CODE GENERATOR]
    F --> G["LDF R2, id3  
MULF R2, R2, #60.0  
LDF R1, id2  
ADDF R1, R1, R2  
STF id1, R1"]
```

Code Optimizer: Improves performance. Reduces: Execution time, Memory usage

Code Generator: Converts IR into: Assembly code, Machine code.

Preprocessing: step that runs before the compiler proper. It prepares the source code by handling special **directives**. It does, Expands macros, Includes header files, Handles conditional compilation, Removes comments (in many languages).

Lexical Analysis: Converts character stream → tokens, Removes whitespace and comments. **Input:** Source code (characters) **Output:** Tokens.

Lexeme: Actual character sequence that matches a token. **Grammar Pattern:** Rule that defines a valid token. patterns are defined using regular expressions.

Parsing (Syntax Analysis): Process of analyzing a sequence of tokens. Checks whether tokens follow the grammar rules of the language. **Input:** Tokens (from lexical analysis) **Output** (used by semantic analysis): Parse tree (structure of the program) or Abstract Syntax Tree (AST) If correct → **builds a parse tree**. If incorrect → **reports a syntax error**. Breaks input into **smaller structural elements**. **Done by-Parser** (a compiler or interpreter component)

Syntax Analysis: Checks against syntactic rules (grammar). **Done by-Parser. Output- Parse tree which Used to generate intermediate code.**

Lexical analysis → **tokens**
Syntax analysis → **structure (parse tree)**

Parser Generator Diagram: **Grammar**-Written in BNF / EBNF, Declarative → defines syntax rules, Not user program code. **Parser Generator**-Input: Grammar + grammar-specific **action code**, Output: Parser code, Decides *what parser should do when a rule is recognized*. **Parser**- Input: Tokens (from lexical analyzer), Checks if tokens follow grammar rules, Detects syntax errors, Builds parse tree.

Intermediate Representation (IR)-Produced after successful parsing. **Machine-independent form of program**, Used for optimization & code generation

Types of parser: **Top-Down Parser:** Starts from start symbol. Tries to generate the input string. Expands non-terminals. **Uses leftmost derivation Builds tree root → leaves.** Simpler, less powerful. Examples: Recursive Descent, LL(1).

Bottom-Up Parser: Starts from input string. Reduces input to start symbol. **Uses rightmost derivation (in reverse)**. Builds tree leaves → root. More powerful (because Uses rightmost derivation in reverse), more complex. Examples: LR, SLR, LALR.

Recursive Descent Parser: Type: Top-down parser. **Idea:** Parsing is done using a set of **recursive functions**. **With Backtracking:** Tries one grammar rule. If it fails → goes back and tries another. Simple but inefficient. May re-parse same input many times. **Without Backtracking (Predictive / LL parser):** Uses lookahead token(s) (e.g., 1 token in LL(1)). Chooses the correct rule directly. No retrying. Faster and efficient. Grammar must be left-factored and no left recursion

Top down approach:

Diagram: S → A B e. A → a b c d. B → e. LL parser. Decision: Which production to use.

Bottom up approach:

Diagram: S ← A B e. A ← a b c d. B ← e. LR parser. Decision: When to reduce.

Classification of Parsers:

```
graph TD
    Parsers --> TopDown[Top down Parsers]
    Parsers --> BottomUp[Bottom up Parse]
    TopDown --> WithBack[With backtracking]
    TopDown --> WithoutBack[Without backtracking]
    WithBack --> Brute[Brute Forcing]
    WithBack --> Recursive[Recursive Descent Parsers]
    WithoutBack --> Predictive[Predictive Parsers]
    Predictive --> LL["LL(1), LL(k)"]
    BottomUp --> Operator[Operator Precedence Parsers]
    BottomUp --> LR[LR Parser]
    LR --> LR0["LR(0)"]
    LR --> SLR["SLR(1)"]
    LR --> LALR["LALR(1)"]
    LR --> CLR["CLR(1)"]
```

An LL(1) parser is a predictive top-down parser that uses one **lookahead symbol (1)** and a parsing table to decide which production rule to apply, without backtracking.

FIRST(X) is the set of terminals that can appear as the first symbol in any string derived from X. **S → a A B C. FIRST(S) = {a}**

e.g. (2): S → ABC. FIRST(S): {a, b}

ε - Means empty. **A->als** means A can be either a or empty. **Case1:** if A is a then ab, therefore, FIRST(S) = a. **Case2:** If A is empty then bc, therefore, FIRST(S) = b. Then FIRST(S) = {a,b}

FOLLOW(X): Which terminals can appear immediately AFTER X in any valid sentence.

Golden rule: FOLLOW(Starting symbol) = \$

C is at the end so FOLLOW(C) = FOLLOW(S)

FOLLOW(A), what comes immediately after A is B, C

Find FIRST(B) = b. But can be ε. Therefore B can be missing. So look FIRST(C) = c. This means FOLLOW(A) = {b, c}

Grammar Notation: BNF (Backus-Naur Form): Uses production rules to define syntax, Widely used in compiler design, Can enforce operator precedence. **EBNF (Extended BNF):**

Supports: Repetition { }, Optional elements [], Grouping (), Simpler and more readable

Syntax Analysis: Compiler verifies token sequence, Checks against syntactic rules (grammar). **Done by-Parser. Output- Parse tree which Used to generate intermediate code.**

Lexical analysis → **tokens**
Syntax analysis → **structure (parse tree)**

Parser Generator Diagram: **Grammar**-Written in BNF / EBNF, Declarative → defines syntax rules, Not user program code. **Parser Generator**-Input: Grammar + grammar-specific **action code**, Output: Parser code, Decides *what parser should do when a rule is recognized*. **Parser**- Input: Tokens (from lexical analyzer), Checks if tokens follow grammar rules, Detects syntax errors, Builds parse tree.

Intermediate Representation (IR)-Produced after successful parsing. **Machine-independent form of program**, Used for optimization & code generation

Flow: Source Code → Lexical Analyzer → Tokens

Tokens + Parser → Parse Tree → Intermediate Representation.

File parsing: process of reading a file and analyzing its contents. Extracts **useful data** according to a **defined format or rules**. Converts raw file data into **structured data**.

Symbol Table: Stores info about identifiers (name, type, scope). Used by lexical analyzer, parser, and front end.

Rest of Front End: Uses parse tree + symbol table, Produces **Intermediate Representation (IR)**.

Semantic Analysis: compiler phase that checks the meaning of the program. It runs after syntax analysis (parsing). **What it checks:** Variables are declared before use, Type checking (int vs float, etc.), Scope rules, Function arguments and return types, No invalid operations (e.g., adding int + string). **Output: Annotated syntax tree.**

Parsing in NLP: process of analyzing the grammatical structure of a sentence. It determines how words relate to each other according to language rules. **Why it's used**-Understand sentence meaning. Used in machine translation, question answering, chatbots.

Rules and the Target is given below

1. E → E + E
2. E → E x E
3. E → id

Context Free Grammar: E → E + E | E x E | id

Left Most Derivation: E ⇒ E + E ⇒ id + E ⇒ id + E x E ⇒ id + id x E ⇒ id + id x id

Right Most Derivation: E ⇒ E + E ⇒ E + E x E ⇒ E + E x id ⇒ E + id x id ⇒ id + id x id

Parse Tree Derivat

Explicitly choose one rule (Ex- E → E + E) and choose another to get the target.

Context-Free Grammar (CFG): Used to describe syntax of programming & formal languages.

CFG is AMBIGUOUS: can be parsed in more than one way. **If more than one parse tree exists** grammar is ambiguous. Look the Ex below.

Context Free Grammar: E → E + E | E x E | id

Left Most Derivation: E ⇒ E + E ⇒ id + E ⇒ id + E x E ⇒ id + E x id ⇒ id + id x id

Right Most Derivation: E ⇒ E + E ⇒ E + E x E ⇒ E + E x id ⇒ E + id x id ⇒ id + id x id

Parse Tree Derivat

Terminals: Basic symbols that **cannot be broken further**. Examples: id, +, *, (,). **Non-terminals:** Syntactic variables representing **sets of strings**. Examples: <expr>, <term>, E, T. **Start Symbol:** Non-terminal that generates the **entire language**. Usually the **first production's LHS**. **Productions:** Form: A → X₁ X₂ ... X_n. Left side (A): a non-terminal. Right side: zero or more terminals/non-terminals. Define how strings are formed.

Virtual Machine (VM): Software-based execution environment that runs programs independent of OS & hardware. Examples: JVM, .NET CLR.

How it works: Source code → Intermediate code (bytecode) → VM → Machine code.

Platform independence: Same bytecode runs on any system with the VM.

JIT (Just-In-Time) compilation: Converts bytecode to native machine code at runtime.

Benefits of Virtual Machine: Write once, run anywhere, Runtime optimizations, Better portability and safety.

Runtime environment: Memory layout + execution context while a program runs.

Memory areas: **Stack:** Function calls, parameters, local variables (auto, LIFO). **Heap:** Dynamic objects/data (new, malloc), flexible lifetime. **Static memory:** Global & static variables, allocated once, exist for entire program run.

Lifetime of Variables: **Static:** Allocated at program start, deallocated at program end → Lifetime = entire program → Stored in static memory
Stack-dynamic: Allocated when function/block is entered. Deallocated when it exits → Stored in stack
Heap-dynamic: Allocated at runtime
Deallocated manually or by garbage collector → Stored in heap.

Memory Management: Manual memory management (C): Programmer allocates & deallocates memory. malloc() allocates, free() deallocates. Fast, but error-prone (leaks, dangling pointers). **Automatic memory management:** Handled by runtime. Garbage Collection (Java, Python). Finds unused objects and frees memory automatically. Safer, but slightly slower due to runtime overhead.

Safety & Security Features: Memory-safe languages - Prevent invalid memory access (no buffer overflows, dangling pointers). **Type safety** - Only compatible types allowed → prevents type errors (e.g., int + string). **Null-safety** - Prevents errors from null or uninitialized references. **Concurrency safety (Rust borrow checker)** - Prevents data races by enforcing safe access to shared data at compile time.

int x = e; **Name(x)** symbolic identifier. **Variable:** A named memory location whose value can change during execution. **Binding(int x = e);** x is associated with a variable. Memory is allocated for x. Binding = association of a name → entity (variable, function, data).. So binding = name → memory location + value + type. **Types of Binding: Static binding:** Done before execution (compile time), fixed. **Dynamic binding:** Done during execution, can change. **Explicit binding:** Programmer specifies it (e.g., int x). **Implicit binding:** Language decides automatically (e.g., Python).

Scope scope of x is the part of the program where x is visible and usable. Scope starts after the declaration. Scope ends at the end of the block. names (identifiers) to refer to data instead of memory addresses. **Not all data has names:** In C, dynamically allocated memory has no name. It is accessed via pointers. The pointer variables are named, not the memory itself. **Scope notation:** S2[x] means x is bound and usable in S2. { S1[x] Scope exists before initialization → variable but uninitialized. Happens in Var type var x = e[x]; Although x is in scope in S1 and e, it S2[x] is not yet initialized.

In C, you can declare a variable without defining it if use it before declare gets garbage value in C **Haskell** allows recursive bindings (lazy evaluation) **Lifetime** of an entity (variable): Time from memory allocation to memory deallocation. **Lifetime of a binding (name):** Time from when a name is associated with the entity to when that association ends. **If Entity lifetime ≠ Binding lifetime** then problem Binding ends before entity → garbage (unused memory still allocated) Binding outlives entity → dangling reference (name refers to freed memory). A binding is active whenever the name can be used to access its entity. A scope is the largest region of the program where bindings do not change.

```
int x = 5;
if (y > 0) {
    int x = 7; // shadows x = 5
    print(x); // prints 7
}
```

Bindings can be temporarily deactivated. This happens due to shadowing. Function calls: Bindings of the caller may be inactive while another function executes. **Static variables:** Storage exists for entire program. Binding may be inactive when the function is not running.

Types of Storage Allocation **Static** Fixed memory location. Allocated before execution, freed after program ends. **Stack** Follows function call / return Automatic allocation & deallocation. **Heap** Allocated at run time, Independent of call structure. **Static Allocation is used for,** Program code, Global variables, Static variables. **Stack Allocation is used for** Function parameters, Local variables **Temporary values.** Why use a Stack? Supports recursive functions, Each call needs its own variables (Not needed in FORTRAN, FORTRAN did NOT allow recursive functions therefore no stack was needed.). Enables memory reuse. Same stack space reused after function returns **Why NOT always use a Stack?** Closures (closure = function + variables it uses from outside its scope. Variables used by the function must stay alive. Variable lifetime may exceed binding lifetime. Stack memory is destroyed after function returns. So closures cannot rely on stack allocation) break stack discipline. Variables may be needed after function returns. Stack memory would be destroyed → invalid access. They require heap allocation. **Stack maintenance is handled by:** Calling sequence (caller side), Subroutine prologue & epilogue (callee side). **Why put as much work as possible in the callee's prologue & epilogue?** To save space and reduce code duplication. A function may be called from many places. If stack setup/cleanup is done in the caller: Same code is repeated at every call site, Program size increases. If done in the callee

(prologue/epilogue): Code is written once. All callers reuse it. Therefore, **Caller:** just jumps to function **Callee:** handles stack setup & cleanup. **Internal Fragmentation.** Wasted space inside allocated blocks. Happens when allocated memory is larger than requested. **External Fragmentation.** Free memory exists but is split into small, non-contiguous blocks. Total free space may be sufficient, but no single block is large enough. **Declaration:** Introduces a name, Specifies its type, Does not allocate storage or give full meaning **Definition:** Fully defines the entity, Allocates storage / gives value (variables), Provides function body (functions). **purpose of a definition** - less declaration is to allow the name to be used, even though the definition will be somewhere else **Common rules** **Declaration before use** → Compiler must know the name and type **Definition before use** → Needed when actual value or body is required. **Why Declaration before use?** Enables one-pass compilation. Function calls know the signature (type, parameters). **Why Definition before use?** Prevents use of undefined variables Ensures values/storage exist when accessed. **Java** relates both of these for classes, fields, and methods But not for local variables (variables must be defined before use). **Explicit Declaration:** when a variable is clearly declared using: A keyword or A type name. **Implicit declaration:** when a variable is created automatically, without an explicit keyword or type;

```
let y = 5; // explicit declaration
x = 5 // implicitly declared
```

Static (Lexical) Scope → Scope is defined by program text (lexical structure). Inner functions can access variables of outer functions, not runtime calls. Compiler can determine scopes at compile time. Identifier bindings are resolved by reading the code. Uses the **most closely nested rule**. **Static Scope:** An identifier is visible: In the scope where it is declared. In all inner (enclosed) scopes. Unless redeclared (shadowed) in an inner scope. To resolve a name (identifier): Check local scope first. Then check statically enclosing scopes. Stop at the first binding found. **Scope hole** occurs in static scoping when: An inner declaration has the same name! shadows the outer declaration. **Static Links:** Used to access non-local variables under static (lexical) scoping. Each **stack frame** has a **static link** pointing to the frame of the **lexically enclosing routine**. **Static Chains:** A static chain is the sequence of static links connecting stack frames. Used to **access non-local variables** under **static (lexical) scoping**. **Dynamic Scope:** No static links are used. Variable reference is resolved by finding the most recent binding on the call stack. Binding may come from unrelated calling code. Cannot determine bindings from program text alone. Makes programs hard to understand, debug, and reason about. Dynamic scope resolves names using the call stack, not lexical structure. **Considered bad design today.**

```
{var a;
function first() { a = 1; } //Static (lexical) scoping →
function second() { var a; first(); } prints 1
a = 2; second(); print(a); } //Dynamic scoping →
prints 2 .
```

Static-first() refers to a in its **lexically enclosing scope** (the outer a). **Dynamic-first()** refers to the **most recent a on the call stack**. **Referencing environment** at runtime is the set of active bindings. It represents the scopes searched (in order) to resolve a name. Name lookup checks these scopes until a binding is found. **First-class function** = function treated like a value (can be returned, stored in variables, passed as arguments). **Lambda Expression:** A lambda expression is an anonymous function (no name). Used to define small functions inline. **Closure = function + its referencing environment.** Created when a function is: Passed as a value, Returned from another function, Stored in a variable. **Why closures are needed?** To support static (lexical) scoping. Referenced variables may outlive their original scope. **Closures don't need for languages with dynamic scoping.** Uses caller's environment instead. This is called **shallow binding**. **Lisp:** Normally dynamic scope, Supports closures when explicitly requested.

Aliasing: (two or more names refer to the same entity (address)). **Benefits:** Enables linked data structures Allows more efficient algorithms. Example: union-find. **Drawbacks:** Makes compiler optimization harder. FORTRAN disallows aliasing between procedure arguments → enabled faster code than C. Updating data via one name affects all aliases. **Polyorphism:** Same name refers to multiple entities. **Static polyorphism:** Resolved at compile time. Example: function/operator overloading **Dynamic polyorphism:** Resolved at run time Example: method overriding (virtual functions) **Overloading (Ex for Polym):** same function name, different parameter types. Very common for **operators** (+ for integers vs reals). **Polyorphism = one name, many entities; Aliasing = many names, one entity.** **Binding time** = moment when a binding or decision is fixed. **Types of binding time:** **Language design time.** Language constructs, Built-in types. **Language implementation time:** I/O behavior, Arithmetic overflow handling (if not specified in language definition). **Static binding** → binding occurs before run time. **Dynamic binding** → binding occurs at run time. Terms **static** and **dynamic** are **coarse-grained** (not very precise). Actual binding may occur at many specific times (compile, link, load, run). **Early binding** → higher efficiency **Late binding** → greater flexibility. **Hidden binding** → Whether bindings shadowed in inner

scopes are still accessible. **Compiled vs Interpreted:** Not a strict distinction. Any language can be implemented either way. **Early-bound languages** (e.g. C): **Easier to compile.** More efficient execution. **Late-bound languages** (e.g. Python): **Easier to interpret.** More flexible at runtime. Languages that are **easy to understand** are usually **easy to compile**. **Effects of easy-to-compile languages:** More good compilers on more machines, Faster and better code, Fewer compiler bugs, Smaller, cheaper, faster compilers, Better error diagnostics. **Variables (int varName = varValue):** Value: Data stored in the variable (RHS of assignment). **Name:** Identifier used to refer to the variable (varName). **Address:** Memory location holding the value (hidden from programmer). **Type:** Determines size + interpretation of stored data (int, etc.). **Aliasing:** Two names refer to the same address → harder to reason/optimize. **Scope & Lifetime:** **Scope:** Where a variable can be accessed (its "reach"). Often limited to a function/block. **Lifetime:** How long the variable's memory exists. Typically during execution of its scope. **C++ static:** **Extends lifetime** (exists for entire program), **Does not change scope** (still only visible in the function). **Language design time** - Available constructs and types. **Language implementation time** - Representation & precision of primitives, Stack layout, Example: size of int in C. **Program writing time** - Choice of algorithms, data structures, names. **Compile time** - Source → machine code mapping. Data structure layout. **Link time** - Bind module imports to referenced modules. **Load time** - Decide exact memory addresses for code/data. **Run time** - Bind variables to values. **Literal:** a fixed value written directly in code. Value is known at compile time. **Binding:** Associates a name → entity (variable, location), Happens once (typically at declaration) **Assignment:** Changes the value stored in the bound entity (existing variable). Can happen many times

```
int x; // binding
x = 3; // assignment
x = 5; // assignment
```

Assignable: An entity whose value can be changed **Rebinding:** changing which entity a name refers to.

```
let x = 3 // binding x → entity1
let x = 5 // rebinding x → new entity
```

Initialization: Creates a new variable, Gives it an initial value, Happens once.

```
int x = 3; // initialization
x = 5; // assignment
```

Pointer: A pointer is a named reference that stores a memory address. The address is numeric. Pointer values can be manipulated using arithmetic.

Why Do We Need Data Types? Help the compiler/interpreter understand data, Prevent invalid operations (type errors), Improve program reliability. **Data Type:** defines: The kind of data (e.g., integer, float). The operations allowed on the data. The memory representation of the data. **Primitive Data Types:** basic built-in types found in almost all languages. **Integer (int)** → whole numbers. **Floating point (float, double)** → real numbers. **Character (char)** → single character. **Derived & Composite Data Types:** combine multiple values into one unit. **Arrays** → collection of same-type elements. **Classes / Objects** → data + methods. **Strings** → sequence of characters **Abstract Data Types (ADT):** Focus on what operations are allowed, not how implemented. Hide internal representation. **Stack** → push, pop **Queue** → enqueue, dequeue. **List, Map.** **Why ADTs?** Improve modularity, Improve abstraction, Make programs easier to maintain. **Type System?** A type system is a set of rules that: Assigns types to program elements, Controls how types interact, prevents invalid or meaningless operations. **Purpose of a Type System:** Detect errors early (often at compile time), Enforce correctness, Improve documentation (code clarity). **Type Errors:** Occur when: An invalid operation is performed on a data type. Incompatible types are used together.

```
int x = "hello"; // ✗ type error.
```

Classification of Type Systems: Static vs Dynamic typing. **Static** → type checking at compile time, Errors detected early, Safer but more verbose, high performance (no need to check type during execution) **Ex: Java, C++, C.** **Dynamic** → Type checking at run time, More flexible (No need to declare types in advance, More adaptable to dynamic data), Errors appear during execution **Ex: Python, javascript.** **Strong vs Weak typing.** **Strong** → Strict rules on how types are used, No implicit unsafe conversions, Type errors are not silently ignored. **Ex: "5" + 3 ✗** type error. **Weak** → allows implicit type conversions, Can lead to unexpected behavior. **Explicit vs Implicit typing.** **Explicit** → programmer specifies types. **Ex: int x = 10;** **Implicit** → compiler infers types. **Ex: x = 10;**

Type checking: process of verifying type correctness. Ensures operations are applied to compatible and valid types, prevents invalid operations on data types. **Compile-Time Type Checking:** Performed before program execution, Errors detected early, Used in statically typed languages. Improves safety and reliability. **Run-Time Type Checking:** Performed during program execution, Common in dynamically typed languages, More flexible, but riskier (errors at runtime), trades safety for flexibility. **Type Safety:** A language is type-safe if it prevents type errors at runtime. **Type-safe languages:** Java, Python. **Less type-safe:** C (due to

pointer misuse). **Real-World Examples:** **Banking systems** → **Static typing** (high safety, reliability) **Rapid prototyping** → **Dynamic typing** (speed, flexibility) **Embedded systems** → **Strong static typing** (predictability, safety).

Expression: combination of operands (values/variables) and operators that produces a value. **Operands** → variables, constants, literals **Operators** → arithmetic, relational, logical, assignment. **Operator arity:** **Unary** (one operand), **Binary** (two operands), **Ternary** (three operands). **Operator Precedence:** Determines the order of evaluation in expressions. Parentheses have the highest precedence. Multiplication/division before addition/subtraction. Assignment has low precedence. **Associativity:** Used when operators have same precedence, Determines evaluation direction. Left-to-right or Right-to-left. Assignment operators are usually right-associative. **Short-Circuit Evaluation:** Applies to logical AND (&&) and OR (||). Second operand may not be evaluated **AND (&&)** If the first condition is false, the result is false. So the second condition is NOT checked. **OR (||)** If the first condition is true, the result is true, So the second condition is NOT checked, Improves performance, Prevents errors (e.g., null checks). **Side Effects:** occurs when an expression changes a variable/state (modify program state during expression evaluation.). Common with increment/decrement (++ , --). Order of evaluation matters and can change results.

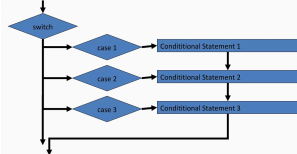
```
x = 5;
y = x++; // side effect: x becomes 6
```

Type Conversion: Implicit conversion (type coercion). Done automatically by the language **May** change type without notice. **Explicit conversion (casting).** Done by the programmer **Uses** cast syntax. This is better because Compiler can warn you about risky casts and No silent, unexpected behavior.

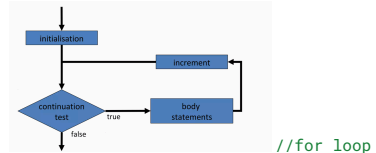
```
int x = 3.5; // implicit → x = 3
int y = (int)3.5; // explicit cast.
```

Control Structures (logic Structures): **three basic control structures:** **Sequence** - Statements execute one after another, Default flow of execution. **Repetition (Loop / Iteration)** - Repeats statements while a condition is true, Examples: **for**, **while**. **Selection (Branching/decision)** - Chooses different execution paths based on a condition. Examples: **if-else**, **switch**. **Flow Control:** describes how execution moves through a program. Determines **which statements run and in what order.** Implemented using **control-structure keywords**. **Statement Block:** groups multiple statements into one unit. Enclosed using curly braces { ... } **if (x > 0) {**
 y = y + 1;
 z = z * 2; //Statement blocks allow multiple statements to execute as a single unit.
}

Nested if Statement: A nested if is an if statement inside another if. Inner if executes only if the outer if is true. **Goto:** allows an unconditional jump to another point in the program. Transfers control directly to a labeled statement. Breaks normal flow of execution. **goto** can make programs hard to read and maintain; generally avoided. **Switch Statement:** A structured alternative to **goto**. Clearer and less confusing. Selects execution based on constant values of an expression. Similar to if-else if-else, but simpler and cleaner. Improves readability and maintainability.



Compares it with **constant values** (case labels). Execute until find **break**. **break** exits the switch and continues after it. **If No break:** Execution continues into the next case. This is called **fall-through**. This execute until a **break** is found or **switch ends**. **Loop:** repeats statements multiple times. **A loop has two parts:** **Body of loop** → statements that repeat. **Control of loop** → condition / counter controlling repetition. **Control of Loop has two parts:** **Entry-Control Loop (Base loop)** - Condition checked before executing body. If condition is false → loop never executes. Examples: **while**, **for**. **Exit-Control Loop (Derived loop)** - Body executed before checking condition. Body executes at least once. Example: **do-while**. **for loop: Use when,** When number of iterations is known. Loop has initialization, condition, update together. Example: counting loops. **for loop has 4 logical parts.** **Initialization** - Executed once at the start. Example: **int i = 0**. **Condition** - Checked before each iteration. Loop runs while condition is true. **Body** - Statements that are repeated. **Update (Increment/Decrement)** - Executed after each iteration. Example: **i++**



While loop: Use when, When number of iterations is unknown. Loop depends on a condition. Example: reading input until condition fails.

Do-While: Loop body executes at least once, Condition is checked after executing the body.

Break Statement: **break** immediately terminates:

for, while, do-while loops. **switch** statement Control jumps to the statement after the loop or switch.

Continue Statement: Goes straight back to the start of a do, while or for loop.

Algorithm: step-by-step procedure to solve a problem. Uses basic **control structures**:

Exception: error condition that disrupts normal program flow. When an exception occurs: Normal execution stops, Program may terminate abnormally.

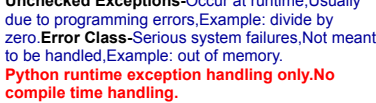
exceptions should be **handled** to maintain normal execution. **Why Exceptions Occur?** Exceptions arise due problem that arises during the

execution of a program. Common reasons: Invalid user input, File not found, Network connection failure, Out of memory (JVM memory exhaustion).

Types of Exceptions(3 type): **Checked Exceptions**-Detected at compile time, Must be handled or declared, Example: file not found

Unchecked Exceptions-Occur at runtime, Usually due to programming errors, Example: divide by zero. **Error Class**-Serious system failures, Not meant to be handled, Example: out of memory.

Python runtime exception handling only.No compile time handling.



All **exceptions and errors** inherit from **Throwable**. **Throwable** is the root of the hierarchy. **Two main branches:** **Error**-Serious failures, Occur at runtime, Not usually handled, Used by **JVM**.

Exception-Can occur at compile time or runtime, Meant to be handled, Used for abnormal conditions in programs.

Errors	Exceptions
1. Impossible to recover from an error	1. Possible to recover from exceptions
2. Errors are of type 'unchecked'	2. Exceptions can be either 'checked' or 'unchecked'
3. Occur at runtime	3. Can occur at compile time or run time
4. Caused by the application running environment	4. Caused by the application itself

Basic Example of Exception

```
class Exception{
public static void main(String args){
try{
//code that may raise exception
}
catch(Exception e){
// rest of the program
}
}
```

// catches both checked and unchecked exceptions. Whether an exception is checked depends on the actual exception thrown, not the catch block. Ex: **Checked:** **IOException**, **SQLException**. **Unchecked:** **NullPointerException**, **ArithmeticException**. **Exception is a superclass, not a specific checked exception.**

Unchecked Ex:

```
int[] arr = {1,2,3};
System.out.println(arr[5]); // Throws
```

ArrayIndexOutOfBoundsException. Occurs at runtime. Unchecked exception (not checked by compiler). **Exception Propagation:** When an exception occurs, it flows up the call stack, Passed from method to method, Handled by the nearest matching catch block. If not caught → program terminates abnormally. **Checked Exceptions & Meaning:** **ClassNotFoundException** → Class cannot be found at runtime. **CloneNotSupportedException** → Object does not implement **Cloneable**. **IllegalAccessException** → Access to a class/member is denied.

InterruptedException → A thread is interrupted during execution. **NoSuchFieldException** → Requested field does not exist.

NoSuchMethodException → Requested method does not exist. **Built-in Exceptions and meaning:** **ArithmeticException** → Error in arithmetic operation (e.g., divide by zero)

ArrayIndexOutOfBoundsException → Array accessed with invalid index.

ClassNotFoundException → Class definition not found at runtime. **FileNotFoundException** → File does not exist or cannot be opened. **IOException** → Input/output operation failed or interrupted.

InterruptedException → Thread interrupted while waiting/sleeping/processing. **NoSuchFieldException** → Specified field does not exist in the class.

User-Defined Exceptions: Used when built-in exceptions are not sufficient. Created by the programmer. Represent custom error conditions. Typically extend the **Exception** class.

Exception Handling: Exception handling manages runtime errors, Prevents abnormal program termination, Improves program robustness and reliability, Allows program to continue execution safely. **Exception Handling Flow:** Problem

Occurs → An error happens during execution.

Create Exception → An exception object is created. **Throw Exception** → Exception is thrown to the runtime system. **Handle Exception** → Nearest matching **catch** handles it. If not handled → program/system failure. **Exception Handling Blocks:** **try**-Contains code that may cause an exception. **catch**-Handles the exception.

finally-Always executes, Used for cleanup (close files, release resources).

Exception Handling Variants: **Multiple Catch Handlers:** Use multiple catch blocks, Handle different exception types separately

Catch-All Handler: Catches any exception. Ex-catch (Exception e). **Cleanup Handler:** Uses **finally**, Always executes, Used for resource cleanup (close files, release memory).

throw-Explicitly creates and raises an exception, Used inside a method.

throws-keyword in method declarations. Declares that a method may pass exceptions to caller, Does not raise the exception.

Functional Language: class of programming languages based on mathematical functions and lambda calculus, where computation is treated as the evaluation of functions. Functions behave like mathematical equations (Same input → same output). Supports **first-class functions**: Functions can be passed as arguments, Returned from other functions, Stored in variables. Uses **recursion instead of loops**. Emphasizes **immutability** (once a variable is set, its value cannot be changed). Reduces **unexpected bugs**. Supports parallel programming, Multiple computations can run simultaneously, Easier due to immutability and no side effects. **No Side Effects:** A function has no side effects if it: Does not change any external state, Does not modify variables, files, or I/O Output depends only on input values.

Lazy evaluation-Expressions evaluated **only when needed**, Improves performance, Allows infinite data structures.

Pure Function: pure function has all three properties together: Immutability-Data is not changed, New values are created instead of modifying existing ones. No Side Effects-Does not modify external state, No changes to variables, files, databases, or I/O. **Referential Transparency** -Function call can be replaced by its result, Same input → same output always. **Referential Transparency Key Characteristics:** Always produces the same output for the same input, Does not modify external state, Does not depend on mutable variables, I/O, or time, Has no side effects.

Referential Transparency Ex-

```
def add(a, b):
return a + b
```

add(2,3) Always evaluates to 5, Can be replaced by 5 anywhere. Referentially transparent

Non-Referential Transparency Ex-

```
def f(a):
return a + x
```

If x changes, result changes. Depends on external state. Not referentially transparent

Importance of Referential Transparency: **Easier Reasoning**-Predictable behavior, Mathematical reasoning possible. **Easier Debugging & Testing**- No hidden state, No unexpected interactions. **Safe Parallel Execution**-No shared mutable state, No race conditions.

Relation to Pure Functions- Pure function-Same input → same output, No side effects. **Therefore, All pure functions are referentially transparent.**

Causes of Loss of Referential Transparency: **Global variables**-Using or modifying shared globals. **Mutable state**-Changing shared data. **I/O operations**-Printing or reading input (because input and output depend on external state and environment). **Random numbers**-**rand()** gives different results. **Time functions**-**time()** changes over time. **Exceptions**- Unpredictable control flow changes.

Referential Transparency: Advantages: Predictable behavior, Easier formal verification, Safe concurrency (no shared mutable state), Enables better compiler optimizations. **Disadvantages:** Hard to model: Input/Output, State changes, Interactive systems.

Do I/O operations preserve referential transparency? Why? No, because input and output depend on external state and environment.

Which programming paradigm emphasizes referential transparency? Functional programming.

Can imperative languages support referential transparency? Partially, by avoiding mutable state and side effects.

Higher-order functions: function that: Takes one or more functions as arguments, and/or Returns a function as its result such as map, filter, and reduce, are commonly used to abstract over common programming patterns and work with collections of data. **Why higher order Important?** Enable functional programming style, Improve code reuse and abstraction, Simplify complex data operations

Higher-Order Functions Advantages: High level of abstraction, Less repetitive code, Easier parallelization, More expressive and concise.

Disadvantages: Harder for beginners, Debugging can be complex.

A language supports higher-order functions if it allows: Functions as first-class values, Functions can be: Assigned to variables, Passed as arguments, Returned from other functions, Stored in data structures.

```
def apply(f, x):
return f(x)
```

```
def square(n):
return n * n
```

```
result = apply(square, 5) # result = 25
```

//Ex for higher order

Advantages of Functional Programming: **Fewer Bugs**-No side effects, No mutable state, Easier reasoning. **Easier Testing**-Pure functions, Output depends only on inputs. **Concurrency & Parallelism**-No shared mutable state, Naturally thread-safe.

Modularity-Small, composable functions, Reusable and maintainable code. **Pure Functional Languages:** Haskell, Clean, Miranda. **Impure / Multi-Paradigm Languages:** Common Lisp, Scheme, OCaml, Scala, F#, **Mainstream languages** like Python, JavaScript, Java (since Java 8).

Closure: A function that remembers variables from its defining scope. Keeps access to those variables even after the scope ends. **Often created when** Functions are returned from other functions

Closure used in: **Encapsulation** → hide internal data, **State preservation** → remember values across calls.

Term	Meaning
First-class function	Functions treated like values
Higher-order function	Function that takes/returns functions
Lambda expression	Anonymous function
Closure	Function with remembered environment
Function composition	Combining functions
Callback	Function passed as argument

callback function? A function passed as an argument to another function to be executed later.

function composition? Combining two or more functions so that the output of one becomes the input of another. **Ordinary function**- Takes data and returns data. **Higher-order function**- Takes functions or returns functions.

Relation between closures and higher-order functions? Closures are often created when higher-order functions return inner functions that capture variables.

Iron triangle(Traditional)- Here scope is fixed, schedule and resources are dependent and adjusted as needed (deliver everything planned, even if it costs more or takes longer.)

Quality driven triangle (Agile)-schedule and resources are fixed, scope is flexible.(deliver the highest quality and most valuable work within fixed time and people limits).

Agile Manifesto(Official declaration)-4 values and 12 principles (Focus on individuals & interactions, Working software over documentation, Customer collaboration over contract negotiation, Responding to change over following a plan)

Customer satisfaction first – Deliver valuable software early and often so customers are happy. **Welcome change** – Even late in the project, accept changes that can improve the product. **Frequent delivery**. **Business & tech work together** – Developers and business people should collaborate daily. **Motivated teams** – Give people the right environment, support, and trust to get the job done. Agile development focuses on achieving **Personal Success** (Agile encourages team ownership, open communication, regular feedback, and personal growth through retrospectives and adaptive roles.) **Technical Success** (Through continuous integration, automated testing, refactoring and incremental delivery of working software.) **Organizational Success** (improve profitability, strengthen market position, enhance customer satisfaction, and build a strong brand).

Properties of agile-Cancel bad projects early, Replace expensive practices, Fast&accurate communication, Regular review & improvement. ,Maintainable & upgradable software. **Scrum key concept**-Time-boxed Iterations (Sprints): Fixed duration cycles, typically 1–4 weeks. **Cross-functional Teams:** Members with different skill sets who can deliver a complete product increment. **Self organizing Teams:** Teams decide how best to accomplish their work without outside micromanagement. **Scrum roles**-**Product Owner** Owns the product vision. Maintains and prioritizes the product backlog. Acts as the primary link between

stakeholders and the development team. Clarifies requirements and makes trade-off decisions. **Scrum Master** Coaches the team on Scrum principles and practices. Removes impediments that block the team. Facilitates Scrum events and ensures adherence to the process. **Development Team**—9 members (excluding PO & SM). Cross-functional may include testers, programmers, architects, DB admins and UI designers. Delivers potentially shippable product increments every sprint. **Scrum phases** **Pre-game Planning**: Define the system, build product backlog. **Product Backlog**—Ordered list of everything that might be needed in the product. Maintained by the Product Owner. Always prioritized so the most valuable work is done first. **Sprint Backlog**—Subset of the product backlog selected for a sprint. Created during sprint planning. Contains tasks broken down by the development team. **Portfolio Planning** → Decide which products to develop, order, and timeline. **Product Planning** → Define product vision, create high-level product backlog, and roadmap. **Release Planning** → Decide scope, date, and budget for releases, make trade-offs. **Sprints (Game)** Iterative cycles with development and enhancement. Each sprint includes: **Sprint Planning** → Select backlog items for the sprint and decide how to deliver them. **Daily Scrum (Stand-up)** → Short daily meeting (~15 min) to share progress, plans and blockers. **Pigs** = Committed team members (developers, Scrum Master, Product Owner), responsible for delivering the sprint goal and actively speak in daily Scrum. **Chickens** = Stakeholders /observers (e.g., managers), interested but not accountable; may attend daily Scrum silently without interrupting. **Rule**: Only pigs talk during the daily Scrum; chickens give feedback after the meeting if needed. **Questions**: What did I accomplish since the last daily scrum? What do I plan to work on by the next daily scrum? What are the obstacles that are preventing me from making progress? **Sprint Review (Sprint Demo)** → Focus on the *product*: inspect and adapt the product with Scrum team, stakeholders, sponsors. Review completed features in the big-picture context. Ensure the best business solution is built. Share updates both ways: stakeholders guide direction, team gains business insights. May include smaller, intra-sprint reviews. **Sprint Retrospective** → Focus on the *process*: Held after Sprint Review, before next Sprint Planning. Scrum team only. Identify ways to improve how the team works. Create actionable improvements for the next sprint **Postgame** Check if requirements are satisfied. Ensure no pending issues remain. Final deliverables include: Working & tested software, Sprint release notes, Test reports, Progress tracked via burn-down charts or velocity charts. **Planning Principles**: You can't make a perfect plan from the start. Do some initial planning, but don't overdo it. Keep your options open until you absolutely must decide. Release small updates often instead of one big release.

A risky concepts early, don't add extra features too soon, refactor often. **Coding**: Customer stays available, follow coding standards, write tests first, pair program, integrate often, share code ownership. **Testing**: Every feature has tests, all tests must pass before release, add tests for bugs, run acceptance tests often and share results. **12 practices**—The planning Game, Small Releases, Metaphor (A shared analogy that explains how the system works, improving communication and design clarity e.g., describing a system as a “conveyor belt”), Simple Design, Testing, Refactoring(Improve code structure without changing behavior to keep it clean, maintainable, and adaptable, while still passing all tests), Pair programming(Two developers work together—one codes (**driver**), one reviews (**navigator**), switching roles often. Improves quality, knowledge sharing, and teamwork). Collective Ownership, Continuous Integration ,40-hour Workweek, On-site Customer, Coding Standards. **Advantages**: Reduce production and post-delivery defects, Overall simplicity, Built-in quality Power to all stakeholders including customers **Disadvantages**: Informal, little documentation, Contact issues.

development→Evolution→servicing→ retire

Legacy systems- rely on older technologies, hardware, software, data, People and Process **Replacement**-costly and risky, try to keep using them. Risks include missing specifications, deep ties to business processes, hidden business rules, new software being late or over budget. **Change**: hard due to inconsistent coding styles, outdated languages, poor documentation, degraded structure, overly optimized code, and data issues. **Management Strategies**: **Scrap** the system and adapt business processes,

Maintain the system as is,

Replace with new system, **Re-engineer** before getting a decision, should get a **business value assessment** from different view points (System end-users; customers; Line; IT and Senior managers). also **System quality assessment** looks at how well the system meets current goals, how efficient and affordable it is to maintain, and the overall quality of the software. **Software maintenance** modify /extend exist components and adding new components, 3 types: **Fault repairs**- fix bugs/vulnerabilities and correct deficiencies in the way meets its requirements. **Environmental adaptation** -Maintain or changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation, **Functionality addition and modification**-Modifying the system to satisfy new requirements. **Reengineering- ●Source code translation**(old programming language to a modern version of the same language/different language) **●Reverse engineering**(The program is analysed and information extracted from it which helps to document its organisation and functionality). **●Program structure improve** (The control structure of the program is analysed and modified to make it easier to read and understand). **●Program modularisation**(Related parts of the program are grouped together, redundancy is removed. In some cases, this stage may involve architectural transformation as discussed). **●Data re-engineering**(The data processed by the program is changed to reflect program changes). **Reengineering cost factors- 1. Quality of the Existing Software**: If the legacy system has poorly written, unstructured, or undocumented code, it will require more effort to understand and improve. High-quality legacy code reduces the time and cost of reengineering. **2. Tool Support**: The availability of reliable tools for analyzing, refactoring, and migrating the software plays a big role. Good tools can automate many parts of reengineering (e.g., code analysis or data migration), reducing manual effort and lowering cost. **3. Extent of Data Conversion**: If the legacy system uses data in formats or structures that are incompatible with modern systems, a lot of effort may be needed to clean, transform, and migrate data, which increases costs. **4. Availability of Expert Staff**: requires knowledge of outdated programming languages or hardware. If experts in the old technologies (like COBOL, Pascal, etc.) are rare or expensive. **Advantages of Re-engineering** ● Reduced risk(high risk in re-developing software)● Reduced cost(The cost of re-engineering is significantly less than the costs of developing new software.) Business process re-engineering (concerned with redesigning business processes to reduce the number of redundant activities and improve process efficiency). **large-scale, often one-time process that typically happens after a system has been in use for a while. Refactoring** is preventative maintenance' that reduces the problems of future change, don't add functionality but concentrate on program improvement. **Continuous, small-scale activity performed during active development. Human Computer Interaction** is about designing, evaluating, and implementing computer systems that are easy and efficient for humans to use, while studying how people interact with them. **Three Paradigms (Ways to Think About HCI)** **Human Factors** focuses on the physical and ergonomic design of computer systems, ensuring that controls, displays, and interfaces match human physical abilities and limitations. **Classical Cognitivism / Information Processing paradigm** views human-computer interaction as a process of information exchange. **Phenomenologically Situated paradigm** considers interaction within the context of real-world situations. It focuses on designing systems that support tasks in their natural environment, such as mobile navigation apps that work seamlessly while a user is walking or driving. **UI (User Interface)**—Visual and interactive elements (screens, buttons, layouts) for user interaction; focuses on aesthetics and usability. **UX (User Experience)** — Overall user journey and feelings when using a product; focuses on understanding user needs, values, abilities, and limits. **Enhancing UX, Why** — Understand user needs and emotions. **What** — Define what users can do with the

product. **How** — Ensure smooth, intuitive interaction.

UX Design Principles: Visibility (simplicity)— Show available features, **Feedback (speed)** — Let users know results of actions, **Constraints (flexibility)** — Prevent mistakes, **Consistency (innovation)**—Keep design uniform, **Affordance (minimalism)** — Design cues show how to use elements. **User Interaction Design**

Part of UX, focusing on how users interact with products. Ensure expectations for new features are realistic by involving users throughout development and providing training. **Interaction Types: Instructing** — Commands to the system, **Conversing** — Dialog-based interaction, **Manipulating** — Direct physical or gesture-based control, **Exploring** — Navigating virtual or physical spaces, **Responding** — System prompts user for input. **Prototyping** Used to test ideas, compare alternatives, clarify unclear requirements, check technical feasibility, evaluate with users, and ensure the design direction fits overall product development.

Fidelity — Level of detail matching the final product. **Low-Fidelity Prototypes** (e.g., storyboards, card-based) **Advantages**: Quick to revise, explore multiple concepts, easy communication, proof of concept. **Disadvantages**: Limited error checking, poor development specs, facilitator-driven, restricted usability testing and navigation. **High-Fidelity Prototypes Advantages**: Nearly complete functionality, fully interactive, user-driven, realistic look/feel, clear navigation, supports testing and marketing. **Disadvantages**: Resource-heavy, slow to modify, inefficient for proof-of-concept, risk of being mistaken for the final product or setting unrealistic expectations.

Code Smell indicates a potential problem in design or structure, signals poor coding practices or design choices. **Increase maintenance cost**. • It also becomes a **risk to reliability**—smelly code tends to hide bugs. • Harder for new developers to understand and contribute. • **Productivity Loss**: Team velocity slows as the system grows more complex. **Common Code Smells: Long Method**-Too much logic in one method, **Large Class**-Does too many unrelated things, **Duplicated Code**-Repeating the same code blocks **Data Clumps**-Frequent grouping of same variables (Introduce Parameter Object), **Feature Envy**- Method uses another class's data, **Switch Statements**- Complex conditional logic

Technical Debt refers to the implied cost of additional work caused by choosing an easy solution now instead of a better approach that would take longer. **Causes of Technical Debt**• Poor naming conventions, No documentation or unit tests, Legacy code without refactoring, Pressure to deliver features quickly, Poor understanding of requirements, Lack of coding standards or best practices, Ignoring refactoring and code reviews, Outdated frameworks. **Types: Design debt** — Poor system structure. **Code debt** — Messy/repetitive code. **Testing debt**—Missing/weak test coverage. **Architectural**—Flawed system foundation. **Infrastructure**—Weak servers, poor security. **People**— Skill gaps, poor knowledge transfer. **Symptoms**: High bug rates, unstable system, Developers avoid certain code areas, Slow performance, hard to scale. **Measurement Tools**: SonarQube, CodeClimate, PMD, ESLint. **Metrics**: Cyclomatic complexity, duplication%, test coverage. **Management**: Track debt like bugs, Measure and prioritize high-impact areas, Refactor regularly (e.g., 10% sprint time), Automate tests, enforce code reviews. **Code Smell vs Technical Debt**

Long-Term Risks: High maintenance cost, fragile code, Low morale, fear to make changes, More bugs/security risks, Can block innovation; may force total rewrite.